Reverse Infinite Scroll Pagination: Master Implementation Handbook

Project: Spring Boot + STOMP WebSocket + PostgreSQL Chat Application

Author: Senior Software Engineering Mentor

Purpose: Self-contained, zero-dependency study guide and technical blueprint for re-building reverse infinite scroll pagination from scratch.

Table of Contents

Section 1 — Why Pagination Exists

1. The Naïve Approach: Unpaginated Chat History

In an unpaginated system, opening a chat sends an HTTP request like `GET /messages` or `GET /messages/private?senderId=1&receiverId;=2`, and the database retrieves **all** rows associated with that conversation:

$$\text{Total Records} = N \quad (\text{where } N \text{ grows indefinitely over time})$$

If a user has been active for 2 years with 40,000 messages in a private conversation:

- The database executes `SELECT * FROM messages WHERE ...` returning all 40,000 rows.

- The JVM instantiates 40,000 ORM entities (`MessageEntity`).
- Jackson serializes 40,000 objects into a single 25 MB JSON string payload.
- The browser parses 25 MB of JSON and appends 40,000 elements into the DOM.

2. Cascading Failure Modes

A. Network Latency & Bandwidth Consumption

Transferring tens of megabytes over mobile or high-latency connections causes severe network bottlenecks. A single HTTP request taking 10–30 seconds leads to perceived application unresponsiveness and high data costs for users.

B. Database & JVM Heap Strain

- **PostgreSQL**: Must scan and load thousands of disk pages into buffer memory to construct the result set.
- **Spring Boot Heap**: Creating 40,000 Java objects per request causes rapid heap consumption. This triggers frequent, long Stop-The-World Garbage Collection (GC) pauses and eventually leads to `java.lang.OutOfMemoryError: Java heap space`.

C. Browser DOM Node Overload

Browsers struggle to compute style layouts and repaint frames when DOM node counts exceed a few thousand. Storing 40,000 message DOM elements causes:

- Excessive client RAM consumption (often exceeding 1.5 GB per tab).
- Severe frame rate drops (jank) during scrolling.
- Mobile browser crashes due to memory limits.

3. User Experience (UX) Reality

When a user opens a chat room, **99% of the time they only want to read the last 15–20 messages** to understand the current context or reply to an ongoing conversation. Fetching historical messages from 6 months ago on initial page load is a wasted compute cost across database, server, network, and browser.

Section 2 — Types of Pagination

| Strategy | Mechanism | Pros | Cons | Ideal Use Case |

| :--- | :--- | :--- | :--- | :--- |

| **Page-Based** | `page=2&size;=20` | Simple UI navigation (`1, 2, 3...`) | Vulnerable to data drift; slow `OFFSET` on high page numbers | Static E-Commerce product listings |

| **Offset-Based** | `LIMIT x OFFSET y` | Direct SQL mapping; flexible offset jump | $O(N)$ DB row scan; data drift in real-time streams | Admin dashboards, internal reporting |

| **Cursor-Based** | `WHERE id < :cursor` | $O(1)$ B-Tree index lookup; immune to real-time data drift | Cannot jump directly to arbitrary page numbers | High-scale real-time feeds (Twitter, Slack) |

| **Infinite Scroll** | Scroll down to fetch next page | Seamless UX; no page numbers | Destroys footer access; high DOM count if un-virtualized | Social media feeds (Instagram, TikTok) |

| **Reverse Infinite Scroll** | Scroll UP to fetch older page | Starts at bottom (newest); matches chat mental model | Complex DOM scroll position restoration math | Chat Applications (WhatsApp, Telegram, Discord) |

Why We Chose Spring Data `Slice` + Reverse Infinite Scroll

For this project (Spring Boot + PostgreSQL + STOMP WebSockets), **Spring Data `Slice` with Offset Pagination and Reverse Infinite Scrolling** was selected based on trade-offs:

1. **Learning Spring Idioms**: Hands-on experience with Spring Data JPA abstractions (`Pageable`, `PageRequest`, `Slice`, `SliceImpl`).

2. **Avoiding `Page` Count Overhead**: `Page` executes an explicit `SELECT COUNT(*)` query before every fetch. `Slice` only queries `LIMIT size + 1`, skipping the count query while providing a `hasNext()` indicator.

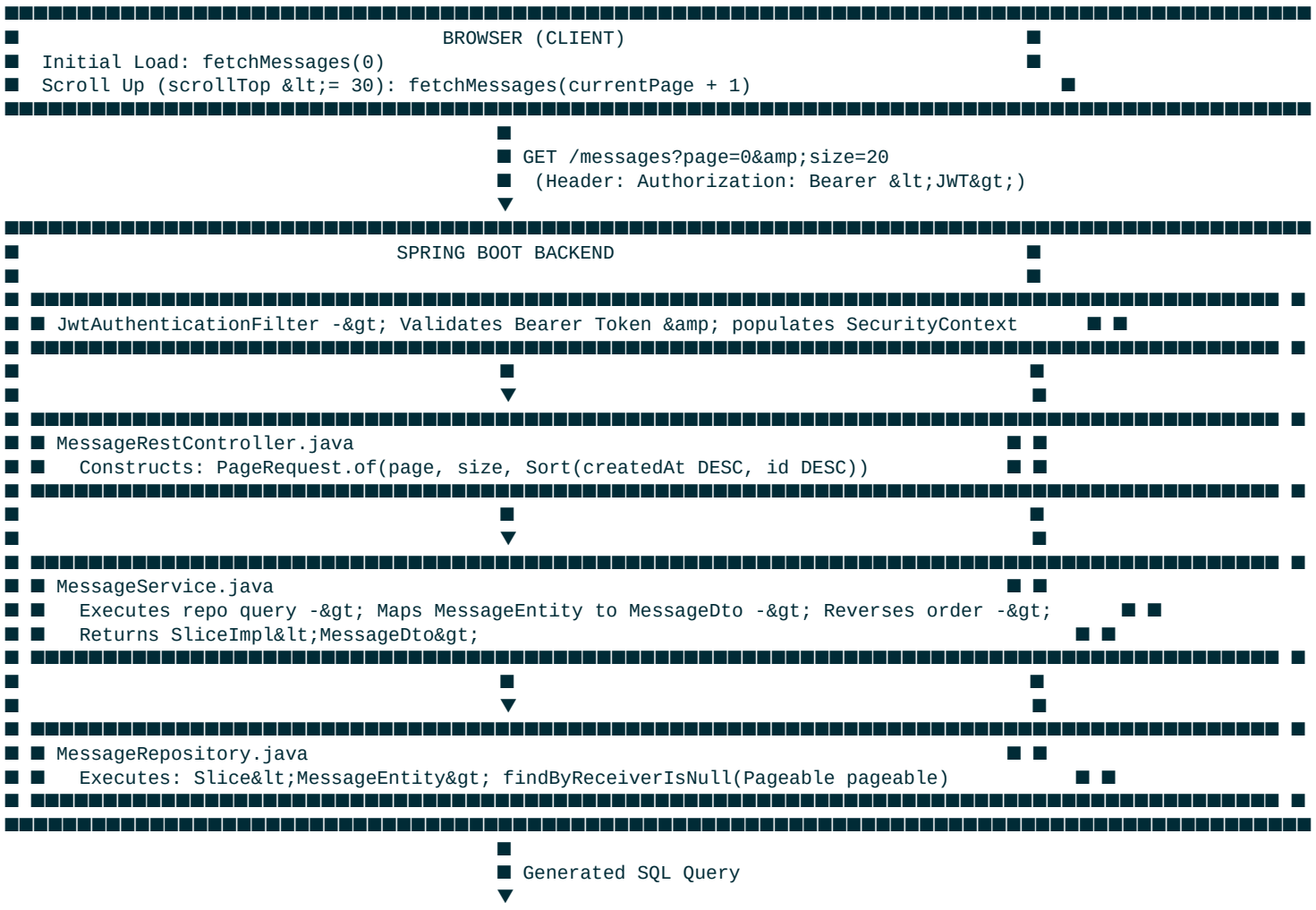
3. **Scale Efficiency**: For conversations with thousands of messages, PostgreSQL B-Tree indexes evaluate `LIMIT 21 OFFSET (page * size)` in under 1 millisecond.

4. **Simple Client-Side Deduplication**: Minor data drift caused by live WebSocket messages arriving while scrolling up is handled in JavaScript using `[data-message-id]` DOM checks.

Section 3 — Project Architecture

The application handles message history via REST and real-time messaging via STOMP WebSockets.

Complete Data Flow Diagram



```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■                                   POSTGRESQL DATABASE                               ■
■                                                                                     ■
■  SELECT m.* FROM messages m WHERE m.receiver_id IS NULL                             ■
■  ORDER BY m.created_at DESC, m.id DESC                                              ■
■  LIMIT 21 OFFSET 0;                                                                 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Section 4 — Repository Layer Deep Dive

File: `MessageRepository.java`

```
package com.chatapp.repository;

import com.chatapp.entity.MessageEntity;
import com.chatapp.entity.MessageStatus;
import com.chatapp.projection.ConversationSummaryProjection;
import com.chatapp.projection.UnreadCountProjection;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Slice;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface MessageRepository extends CrudRepository&lt;MessageEntity, Long&gt; {

    // Public Chat Pagination (Derived Query Method)
    Slice&lt;MessageEntity&gt; findByReceiverIsNull(Pageable pageable);

    // Private Conversation Pagination (JPQL Query)
    @Query("""
        SELECT m
        FROM MessageEntity m
        WHERE
            (m.sender.userId = :user1Id
             AND m.receiver.userId = :user2Id)
        OR
            (m.sender.userId = :user2Id
             AND m.receiver.userId = :user1Id)
    """)
    Slice&lt;MessageEntity&gt; findPrivateConversation(
            @Param("user1Id") Long user1Id,
            @Param("user2Id") Long user2Id,
            Pageable pageable
    );

    // ... existing unread count &amp; conversation summary queries ...
}
```

Key Technical Concepts Explained

1. `CrudRepository` **vs** `JpaRepository`:

- `MessageRepository` extends `CrudRepository`.

- Spring Data evaluates method parameters dynamically. Adding a `Pageable` parameter to *any* query method automatically activates pagination regardless of whether the interface extends `CrudRepository`, `PagingAndSortingRepository`, or

`JpaRepository`.

2. `Slice` **vs** `Page`:

- `Page` executes two SQL statements: the data fetch query AND `SELECT COUNT(*) FROM messages ...`.

- `Slice` executes **only one query**, requesting `size + 1` rows (e.g., 21 rows for a requested page size of 20).

- If PostgreSQL returns 21 rows, Spring Data sets `slice.hasNext() = true` and trims the 21st item from the content list. If PostgreSQL returns 20 or fewer rows, Spring Data sets `slice.hasNext() = false`.

3. **Dynamic Sorting with** `Pageable`:

- In `findPrivateConversation`, notice we removed `ORDER BY m.createdAt` from the hardcoded JPQL string.

- When a `Pageable` containing `Sort.by(Sort.Direction.DESC, "createdAt").and(Sort.by(Sort.Direction.DESC, "id"))` is passed as a parameter, Spring Data JPA dynamically appends `ORDER BY m.created_at DESC, m.id DESC` to the SQL generated at runtime.

Section 5 — Service Layer Design

File: `MessageService.java`

```
public Slice&lt;MessageDto&gt; getHistory(Pageable pageable) {
    // 1. Fetch 21 entities from DB (ordered DESC by DB)
    Slice&lt;MessageEntity&gt; sliceResult = messageRepository.findByReceiverIsNull(pageable);

    // 2. Map Entity -&gt; DTO
    List&lt;MessageDto&gt; messages = new ArrayList&lt;&gt;();
    for (MessageEntity m : sliceResult.getContent()) {
        MessageDto msgDto = buildClientMessageDto(m);
        messages.add(msgDto);
    }

    // 3. Reverse elements in-memory so slice reads oldest -&gt; newest chronologically
    Collections.reverse(messages);

    // 4. Wrap in SliceImpl preserving original pageable and hasNext metadata
    return new SliceImpl&lt;&gt;(messages, pageable, sliceResult.hasNext());
}

public Slice&lt;MessageDto&gt; getPrivateHistory(Long senderId, Long receiverId, Pageable pageable) {

    activeChatTracker.getActiveChats().put(senderId, receiverId);

    // Update status of DELIVERED messages to READ
    List&lt;MessageEntity&gt; deliveredMessages =
            messageRepository.findBySenderUserIdAndReceiverUserIdAndStatus(
                    receiverId,
                    senderId,
                    MessageStatus.DELIVERED
            );

    for (MessageEntity message : deliveredMessages) {
        message.setStatus(MessageStatus.READ);
        MessageEntity savedMessage = messageRepository.save(message);
        MessageDto dto = buildClientMessageDto(savedMessage);

        messagingTemplate.convertAndSend(
                "/topic/user/" + dto.getSenderId(),
                dto
        );
```

```
    }

    // Fetch paginated slice
    Slice&lt;MessageEntity&gt; sliceResult = messageRepository.findPrivateConversation(senderId, receiverId, pageable);

    List&lt;MessageDto&gt; messages = new ArrayList&lt;&gt;();
    for (MessageEntity m : sliceResult.getContent()) {
        MessageDto msgDto = buildClientMessageDto(m);
        messages.add(msgDto);
    }

    // Reverse elements in-memory
    Collections.reverse(messages);

    return new SliceImpl&lt;&gt;(messages, pageable, sliceResult.hasNext());
}
```

Architectural Decisions

1. **Why fetch in `DESC` order but reverse to `ASC` in Service?**

- **Database Order (`DESC`)**: Page 0 (`OFFSET 0`) must retrieve the **newest** 20 messages. If we sorted `ASC` in PostgreSQL, Page 0 would retrieve messages from 2 years ago.

- **In-Memory Reversal (`Collections.reverse`)**: A chat window renders messages top-to-bottom in chronological order (oldest at top, newest at bottom). Reversing the 20 DTOs inside `MessageService` delivers `response.content` in chronological reading order for the frontend.

2. **Why `SliceImpl` is Required**:

- Mapping `MessageEntity` to `MessageDto` produces a plain Java `List`, which strips out pagination metadata (`hasNext`, `number`, `size`).

- `new SliceImpl<>(messages, pageable, sliceResult.hasNext())` reconstructs a `Slice` holding both the reversed DTO list and the original pagination metadata.

Section 6 — Controller & REST API Contract

File: `MessageRestController.java`

```
package com.chatapp.controller;

import com.chatapp.dto.MessageDto;
import com.chatapp.projection.UnreadCountProjection;
import com.chatapp.service.MessageService;
import com.chatapp.service.UserService;
import lombok.AllArgsConstructor;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Slice;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.security.Principal;
import java.util.List;

@AllArgsConstructor
@RestController
```

```
public class MessageRestController {

    private final MessageService messageService;
    private final UserService userService;

    @GetMapping("/messages")
    public Slice&lt;MessageDto&gt; getMessages(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {

        Sort sort = Sort.by(Sort.Direction.DESC, "createdAt")
                .and(Sort.by(Sort.Direction.DESC, "id"));

        Pageable pageable = PageRequest.of(page, size, sort);

        return messageService.getHistory(pageable);
    }

    @GetMapping("/messages/private")
    public Slice&lt;MessageDto&gt; getPrivateMessages(
            @RequestParam Long senderId,
            @RequestParam Long receiverId,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {

        Sort sort = Sort.by(Sort.Direction.DESC, "createdAt")
                .and(Sort.by(Sort.Direction.DESC, "id"));

        Pageable pageable = PageRequest.of(page, size, sort);

        return messageService.getPrivateHistory(senderId, receiverId, pageable);
    }
}
```

JSON Response Structure

When Spring Boot serializes `SliceImpl` via Jackson, the HTTP response payload structured like this is returned:

```
{
  "content": [
    {
      "id": 101,
      "senderId": 2,
      "receiverId": 1,
      "content": "Older message in this slice",
      "createdAt": "2026-07-21T13:50:00",
      "status": "READ",
      "userName": "john"
    },
    {
      "id": 102,
      "senderId": 1,
      "receiverId": 2,
      "content": "Newer message in this slice",
      "createdAt": "2026-07-21T14:00:00",
      "status": "READ",
      "userName": "ayush"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": { "sorted": true, "unsorted": false, "empty": false },
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": false,
```

```
  "first": true,
  "size": 20,
  "number": 0,
  "numberOfElements": 2,
  "empty": false
}
```

Section 7 — Frontend State Management

File: `app.js`

```
let selectedUserId = null;
let currentPage = 0;
let isLoadingMessages = false;
let hasMoreMessages = true;
const PAGE_SIZE = 20;
```

State Variable Reference

1. `selectedUserId`:

- `null` when Public Chat is open.

- `Long` (partner's `userId`) when a private conversation is open.

- Determines which API URL to fetch (`/messages` vs `/messages/private`).

2. `currentPage`:

- Zero-indexed integer tracking the currently loaded page number.

- Incremented when fetching older messages (`currentPage + 1`).

- Reset to `0` when opening a new chat or switching conversations.

3. `hasMoreMessages`:

- Boolean flag indicating whether older historical messages exist in PostgreSQL.

- Derived from `!data.last` (or `data.hasNext`).

- When `false`, the scroll listener ignores scroll events to avoid firing redundant requests.

4. `isLoadingMessages`:

- Mutex lock flag preventing duplicate in-flight network requests.

- Set to `true` at the start of `fetchMessages()` and reset to `false` in `.finally()`.

5. `PAGE_SIZE`:

- Fixed constant (`20`) matching backend default page size.

Section 8 — `fetchMessages()` Line-by-Line Breakdown

```
function fetchMessages(page = 0) {
```

```
// 1. Mutex lock: Ignore call if a request is already in-flight
if (isLoadingMessages) return;
isLoadingMessages = true;

const chatWindow = document.getElementById("messages");
const myId = localStorage.getItem("userId");

// 2. Construct dynamic endpoint URL
const url = selectedUserId == null
    ? `http://localhost:8080/messages?page=${page}&amp;size=${PAGE_SIZE}`
    : `http://localhost:8080/messages/private?senderId=${myId}&amp;receiverId=${selectedUserId}&amp;page=${page}&am

// 3. Record total scrollable height BEFORE adding new nodes to DOM
const oldScrollHeight = chatWindow.scrollHeight;

fetch(url, {
    headers: {
        Authorization: `Bearer ${token}`
    }
})
    .then(response =&gt; response.json())
    .then(data =&gt; {
        const messages = data.content || [];

        // 4. Fallback check for hasNext vs !last (Jackson serialization safety)
        hasMoreMessages = data.hasNext !== undefined ? data.hasNext : !data.last;
        currentPage = page;

        if (page === 0) {
            // Initial Page 0: Clear container and append items
            chatWindow.innerHTML = "";
            let lastSender = null;

            for (const msg of messages) {
                // Deduplicate using dataset ID
                if (document.querySelector(`[data-message-id="${msg.id}"]`)) continue;

                const messageDiv = createMessageElement(msg);
                if (lastSender !== msg.userName) {
                    messageDiv.style.marginTop = "20px";
                }
                lastSender = msg.userName;
                chatWindow.appendChild(messageDiv);
            }

            // Scroll to bottom of chat for Page 0
            chatWindow.scrollTop = chatWindow.scrollHeight;
        } else {
            // Page &gt; 0: Prepend older messages to top using DocumentFragment
            const fragment = document.createDocumentFragment();
            let lastSender = null;

            for (const msg of messages) {
                if (document.querySelector(`[data-message-id="${msg.id}"]`)) continue;

                const messageDiv = createMessageElement(msg);
                if (lastSender !== msg.userName) {
                    messageDiv.style.marginTop = "20px";
                }
                lastSender = msg.userName;
                fragment.appendChild(messageDiv);
            }

            // Insert all older messages in one layout operation
            chatWindow.insertBefore(fragment, chatWindow.firstChild);

            // 5. Restore scroll position to prevent view jumping!
            chatWindow.scrollTop = chatWindow.scrollHeight - oldScrollHeight;
        }

        if (selectedUserId != null &amp;&amp; page === 0) {
```

```
                loadConversations();
            }
        })
        .catch(error =&gt; console.error("Error loading messages:", error))
        .finally(() =&gt; {
            // Unlock mutex lock when HTTP cycle completes
            isLoadingMessages = false;
        });
}
```

Section 9 — Scroll Event Listener Mechanics

```
const messagesContainer = document.getElementById("messages");
if (messagesContainer) {
    messagesContainer.addEventListener("scroll", function () {
        if (messagesContainer.scrollTop &lt;= 30 &amp;&amp; hasMoreMessages &amp;&amp; !isLoadingMessages) {
            fetchMessages(currentPage + 1);
        }
    });
}
```

Why `scrollTop <= 30`?

- Setting the threshold to `30px` instead of `0px` triggers the next fetch slightly *before* the user hits the exact top pixel. This produces a smooth, continuous scrolling experience without hitches.

The Trigger Condition

1. `messagesContainer.scrollTop <= 30`: User has scrolled near the top boundary of `#messages`.
2. `hasMoreMessages === true`: Backend reported that older records exist in PostgreSQL.
3. `!isLoadingMessages`: No pagination request is currently in-flight.

Section 10 — Scroll Position Preservation (The Math)

When prepending older messages to the top of a scroll container, the DOM element's total height expands upwards. Without scroll adjustment, the viewport remains pinned to `scrollTop = 0`, snapping the user's eyes to the oldest prepended message.

Mathematical Formula

$$\Delta H = H_{\text{new}} - H_{\text{old}}$$

$$\text{scrollTop}_{\text{restored}} = \Delta H = H_{\text{new}} - H_{\text{old}}$$

Where:

- H_{old} (`oldScrollHeight`): `chatWindow.scrollHeight` recorded *before* inserting new DOM nodes.
- H_{new} (`chatWindow.scrollHeight`): Expanded `scrollHeight` recorded *after* prepending new DOM nodes.

Numerical Example

1. **Before Prepending**:

- `chatWindow.clientHeight` = 600px

- H_{old} = 1000px

- `scrollTop` = 0px (user scrolled to top)

2. **Prepend 20 Older Messages**:

- 20 messages added $\rightarrow$ adds 800px of content height.

- H_{new} = 1800px.

3. **Scroll Restoration Execution**:

$$\text{scrollTop} = 1800\text{px} - 1000\text{px} = 800\text{px}$$

4. **Visual Result**: The viewport instantly anchors to `800px`, keeping the user focused exactly on the message they were viewing before prepending.

Section 11 — Framework & Library Concepts

1. Spring Data JPA Core Objects

- `Pageable`: Interface defining pagination requests (`pageNumber`, `pageSize`, `Sort`).
- `PageRequest`: Standard implementation of `Pageable` instantiated via `PageRequest.of(page, size, sort)`.
- `Slice`: Interface representing a subset of data with `hasNext()` / `isLast()` metadata without total count overhead.
- `SliceImpl`: Concrete implementation of `Slice` used to return DTO slices from service methods.

2. Jackson Property Introspection Rules

- Jackson inspects getters starting with `get...()` and `is...()`.
- Boolean methods starting with `is...()` (e.g., `isLast()`) are serialized as `"last": false`.
- Boolean methods starting with `has...()` (e.g., `hasNext()`) are ignored by default Jackson configuration unless explicitly annotated with `@JsonProperty`.

Section 12 — Bugs We Faced & Root Cause Analysis

Bug 1: False Suspect — Container Overflow Hypothesis

- **Symptom**: On initial load, only 20 messages appeared, and scrolling up did not fetch page 1.
- **Initial Hypothesis**: Assumed CSS flexbox or `#messages` container lacked `overflow-y: scroll`, preventing scrollbar creation.
- **Investigation**: Inspected `style.css` and ran console height checks (`scrollHeight = 1788px`, `clientHeight = 600px`).
- **Resolution**: Rule out CSS. `#messages` was overflowing properly and scroll events were actively firing.

Bug 2 (CRITICAL): `data.hasNext` evaluated to `undefined`

- **Symptom**: Browser console logs showed:

`text

Messages returned: 20

data.hasNext: undefined

updated hasMoreMessages: undefined

condition (scrollTop <= 30 && hasMoreMessages && !isLoadingMessages): false

`

- **Root Cause**: `MessageService` returned `SliceImpl`. Jackson serialized `isLast()` as `"last": false`, but ignored `hasNext()` because it started with `has...`. Therefore, `data.hasNext` was `undefined` in JavaScript.
- **Debugging Process**: Added explicit `[PAGINATION LOG]` statements inside `fetchMessages()` logging every state mutation and server response payload.
- **Fix**:

`javascript

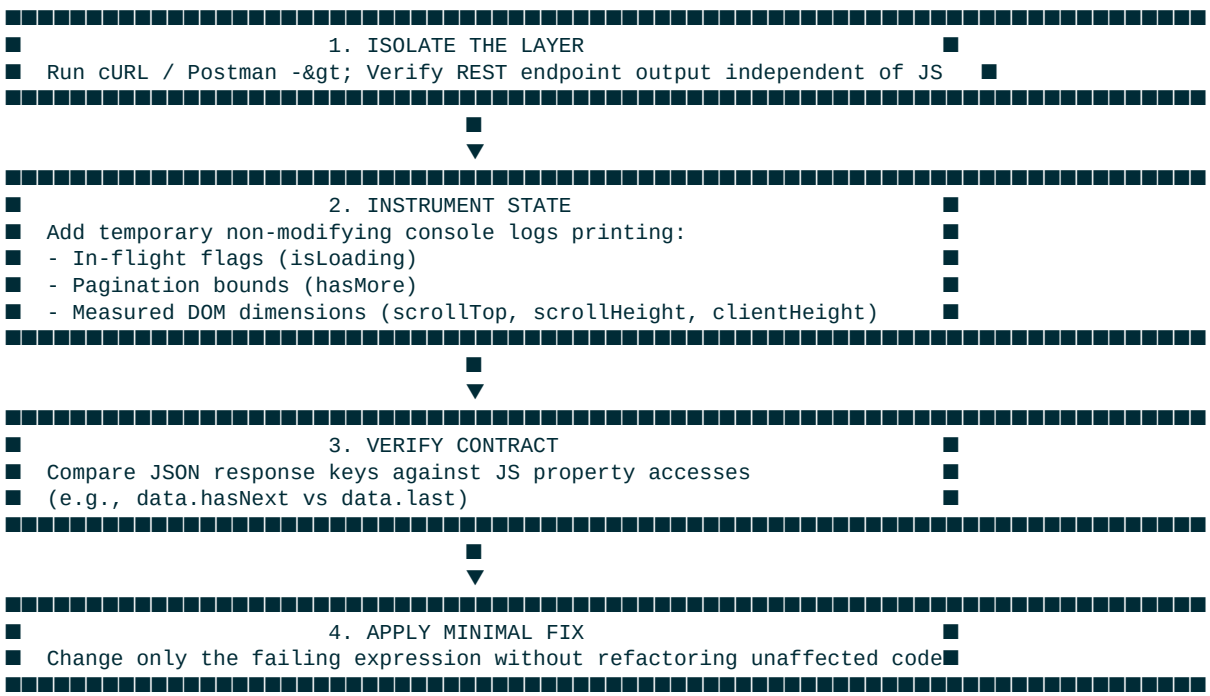
hasMoreMessages = data.hasNext !== undefined ? data.hasNext : !data.last;

`

Since `data.last` was `false`, `!data.last` evaluated to `true`, enabling `hasMoreMessages` and allowing page 1 requests to fire.

Section 13 — Senior Engineering Debugging Methodology

When facing async pagination bugs in real-time web applications, follow this 4-step framework:



Section 14 — Comprehensive Testing Checklist

Backend Verification

- [x] Endpoint `GET /messages?page=0&size;=20` returns HTTP 200 with 20 items.
- [x] Endpoint `GET /messages/private?senderId=1&receiverId;=2&page;=0&size;=20` returns HTTP 200.
- [x] Multi-column sort `createdAt DESC, id DESC` breaks ties deterministically.

Frontend Verification

- [x] Chat opens scrolled to the very bottom on initial Page 0 load.
- [x] Scrolling up to `scrollTop <= 30` triggers fetch for `page = 1`.
- [x] Older messages prepend seamlessly without view jumping.
- [x] Duplicate messages are ignored via `[data-message-id]` check.
- [x] Fast scrolling does not fire duplicate parallel HTTP requests (`isLoadingMessages` mutex lock).

Edge Cases

- [x] Empty conversations return `content: []`, `last: true` without errors.
- [x] Short conversations (< 20 messages) disable further scroll requests (`last: true`).

Section 15 — Final Implementation Summary

Complete System Interaction Diagram

```
[User Clicks Chat]
        ■
        ▼
openPrivateConversation(partnerId)
        ■
        ▼
fetchMessages(page = 0)
        ■
        ■■■■ GET /messages/private?senderId=1&amp;receiverId=2&amp;page=0&amp;size=20
        ■           ■
        ■           ▼
        ■    MessageRestController -&gt; PageRequest.of(0, 20, Sort DESC)
        ■           ■
        ■           ▼
        ■    MessageService -&gt; messageRepository.findPrivateConversation()
        ■           ■
        ■           ▼
        ■    PostgreSQL -&gt; SELECT ... LIMIT 21 OFFSET 0
        ■           ■
        ■           ▼
        ■    Returns SliceImpl -&gt; Reverse List -&gt; JSON Response ("last": false)
        ■
        ▼
Frontend receives JSON:
  - content: 20 DTOs (oldest -&gt; newest)
```

```
  - hasMoreMessages = !data.last (true)
  - Clear #messages -&gt; Append 20 elements -&gt; scrollTop = scrollHeight (Bottom)

[User Scrolls Up -&gt; scrollTop &lt;= 30]
        ■
        ▼
Scroll Listener triggers fetchMessages(page = 1)
        ■
        ■■■■ GET /messages/private?senderId=1&amp;receiverId=2&amp;page=1&amp;size=20
        ■           ■
        ■           ▼
        ■    PostgreSQL -&gt; SELECT ... LIMIT 21 OFFSET 20
        ■           ■
        ■           ▼
        ■    Returns SliceImpl -&gt; Reverse List -&gt; JSON Response
        ■
        ▼
Frontend receives JSON:
  - Record oldScrollHeight
  - Prepend 20 older elements to top via DocumentFragment
  - Restore scroll position: scrollTop = newScrollHeight - oldScrollHeight
  - User continues reading seamlessly!
```